SQL Injection Detection using Machine Learning

A Internship Report submitted in partial fulfillment of the requirements for the award of the degree of

MASTER OF COMPUTER APPLICATIONS

in

COMPUTER APPLICATIONS

By

Pritesh Kumrawat

(205124070)

DEPARTMENT OF COMPUTER APPLICATIONS
NATIONAL INSTITUTE OF TECHNOLOGY
TIRUCHIRAPALLI-620015

JULY 2025

BONAFIDE CERTIFICATE

This is to certify that the project titled **SQL Injection Detection using Machine Learning** is a bonafide record of the work done by

PRITESH KUMRAWAT

(205124070)

in partial fulfillment of the requirements for the award of the degree of **Master of Computer Applications** of the **NATIONAL INSTITUTE OF TECHNOLOGY, TIRUCHIRAPPALLI,** during the year 2024-2025.

Dr. Michael Arock
Guide

Dr. S.Domnic
Head of the Department

Project Viva-voice held on _____

Internal Examiner

External Examiner

ABSTRACT

This internship project report outlines the development of a machine learning-based system for detecting SQL Injection (SQLi) attacks — a prevalent and severe security vulnerability in web applications. The proposed solution uses a Random Forest classifier trained on a dataset of SQL queries that are normalized through custom pattern extraction. These patterns capture common injection techniques such as tautologies, SQL keywords, and encoded characters, enabling the model to effectively distinguish between safe and malicious queries.

The detection model is integrated into a Flask-based web application that provides a real-time interface for users to input queries and receive classification results. An administrator panel is included for monitoring query logs, viewing safe vs. attack statistics, and exporting PDF reports using xhtml2pdf. To enhance usability and portability, the system is wrapped in a PyWebView desktop environment, eliminating the need for browser-based deployment.

This report presents the complete development process, from data preprocessing and model training to web interface design and implementation. The result is a lightweight, user-friendly tool that offers intelligent protection against SQL Injection threats and serves as a solid foundation for further advancements in application-level security.

Through this project, key skills in Python programming, data preprocessing, machine learning, and secure web development were applied and strengthened. The experience also provided valuable insights into practical software deployment, real-world data handling, and the integration of cybersecurity principles into user-friendly applications. This work serves as a meaningful contribution to building safer web ecosystems.

ACKNOWLEDGEMENTS

I express my sincere thanks to **Dr. G. Aghila**, Director, National Institute of Technology, Tiruchirappalli, for permitting me to undertake this project.

I wish to record my sincere thanks to **Dr. S.Domnic**, Head, Department of Computer Applications, National Institute of Technology, Tiruchirappalli I would like to extend my deep gratitude to my internal guide, **Dr. Michael Arock,** Department of Computer Applications, National Institute of Technology, Tiruchirappalli for his whole hearted assistance not only for the duration of project but for the entire duration of my course. I would like to thank him for duly evaluating my progress and encouraging me.

Finally, I would like to express my regards for all the faculty members of Department of Computer Applications, friends and others involved in this project, directly or indirectly.

Pritesh Kumrawat

(205124070)

TABLE OF CONTENTS

Title	Page
ABSTRACT	3
ACKNOWLEDGEMENTS	4
TABLE OF CONTENTS	5
CHAPTER 1 INTRODUCTION	
1.1 Introduction about the Project	6
CHAPTER 2 PROBLEM DEFINITION AND DESCRIPTION	
2.1 Problem Definition	7
2.2 Problem Description	7
CHAPTER 3 SYSTEM ANALYSIS & DESIGN	
3.1 Package Selected	8
3.2 Resources Required	8
3.3 Data Flow Diagrams	9
3.4 I/O Form Design	9
3.5 Table Design	10
CHAPTER 4 IMPLIMENTATION & TESTING	
4.1 Hardware and Software Requirement	11
4.2Installation Procedure	11
4.3 Special Features of Python	11
4.4 Screenshots	12
4.5 Unit Testing	15
4.6 Validation Testing	15
4.7 Experimental Results	15
CHAPTER 5 CONCLUSION AND FUTURE ENHANCMENTS	
5.1 Summery of Achievements	16
5.2 Challenges faced during Project	17
5.3 Future Enhancement	18
REFERENCES	19
APPENDIX	20

CHAPTER 1

INTRODUCTION

1.1 Introduction about the Project

SQL Injection (SQLi) is one of the most persistent and dangerous vulnerabilities in web applications today. It enables an attacker to interfere with the queries that an application makes to its database. By injecting malicious SQL statements into an entry field, attackers can gain unauthorized access to data, modify or delete records, and, in some cases, take full control of the database server. Despite efforts to secure applications using parameterized queries and input sanitization, SQL injections remain a common and critical threat due to poor coding practices or overlooked input validation.

The goal of this internship project is to design and implement a system that automatically detects SQL injection attempts using **machine learning techniques**. Instead of relying solely on manually defined rules or static filters, this system uses a **Random Forest classifier** trained on a data set of labeled SQL queries to differentiate between safe and malicious ones. The machine learning model can generalize and detect new or slightly modified attacks more effectively than traditional rule-based systems.

Before training, all input queries undergo **preprocessing using a pattern extraction module**. This step helps to normalize and tokenize the queries by detecting common SQL keywords, tautologies, logical operators, special characters, and encoded payloads often used in injection attacks. By converting SQL queries into a consistent pattern-based representation, the model can better learn the difference between legitimate and harmful queries.

The project includes a **Flask-based web interface** that allows users to enter SQL queries and view real-time prediction results from the trained model. For administrative purposes, the system includes a **secure admin dashboard**, where all processed queries are logged along with timestamps and classification results. Administrators can view summary statistics and export query logs as **PDF reports** using the `xhtml2pdf` library.

To increase portability across different platforms, the entire application is packaged into a **PyWebView desktop GUI**. This allows users to interact with the system as a native desktop application without needing to open a browser or manually host the server, providing a more polished and accessible user experience.

This project showcases a practical application of data science and cybersecurity, involving data preprocessing, model training, web development, and deployment using Python. The result is a lightweight, intelligent tool capable of detecting SQL injection attempts in real time and contributing to improved application security.

CHAPTER 2

PROBLEM STATEMENT AND DESCRIPTION

2.1 Problem Statement

SQL Injection (SQLi) is a critical vulnerability in web applications that allows attackers to manipulate SQL queries and gain unauthorized access to data. Traditional rule-based detection methods often fail to detect modern and obfuscated attack patterns. There is a need for a smarter, adaptive system that can accurately classify SQL queries and prevent such attacks in real time.

2.1 Problem Description

The core of the problem lies in the dynamic and unpredictable nature of SQL Injection attacks. Attackers can craft queries using logical operators, SQL keywords, comment injections, and encoding schemes to bypass standard validation mechanisms. Manually coding filters for every possible variant is nearly impossible and often leads to high false positives or missed detections.

To tackle this, the system begins by preprocessing raw SQL queries using a pattern extractor that replaces specific values with placeholders. This helps generalize the structure of queries, allowing the model to focus on behavioral patterns rather than hardcoded values.

The cleaned and labeled dataset is then used to train a Random Forest classifier — a robust machine learning algorithm known for handling imbalanced data and noisy features. The trained model is integrated into a Flask web application with a user-friendly frontend that allows users to input queries and receive real-time classification results.

An administrative dashboard supports log monitoring, statistical visualization, and PDF report export. For portability, the entire system is wrapped in a PyWebView desktop interface, allowing it to run as a standalone application without requiring browser deployment.

This system provides a practical, intelligent solution to detecting and preventing SQL Injection, improving the overall security posture of web-based applications.

CHAPTER 3

SYSTEM ANALYSIS AND DESIGN

3.1 Package Selected

The project utilizes several powerful Python libraries for functionality:

- **Flask:** For building the web interface.
- **Scikit-learn:** To implement the machine learning model using Random Forest.
- **joblib:** For saving and loading the trained model and vectorizer.
- **CountVectorizer (from sklearn):** To convert SQL queries into numerical format for training.
- **PyWebView:** To wrap the Flask app into a desktop GUI.
- **xhtml2pdf:** For exporting logs and reports to PDF.

These packages were selected for their performance, ease of use, community support, and compatibility with cross-platform development.

3.2 Resources Required

- **Human Resources:**
  - Developer (ML, Web, GUI integration)
  - Supervisor (for guidance and review)

- **Environmental Resources:**
  - Laptop with Python 3.x installed
  - Code editor (VS Code or PyCharm)
  - Windows or Linux OS
  - Internet connection for downloading libraries

3.3 Data Flow Diagram

The system follows a basic flow:

- **User Input:** A SQL query is entered by the user.
- **Preprocessing:** The query is normalized using pattern extraction.
- **Model Prediction:** The trained Random Forest model classifies the query.
- **Result Display:** Safe or Attack is shown on the UI.
- **Logging:** The query and prediction are saved to a log file.
- **Admin Dashboard:** Admin can view statistics and export reports.

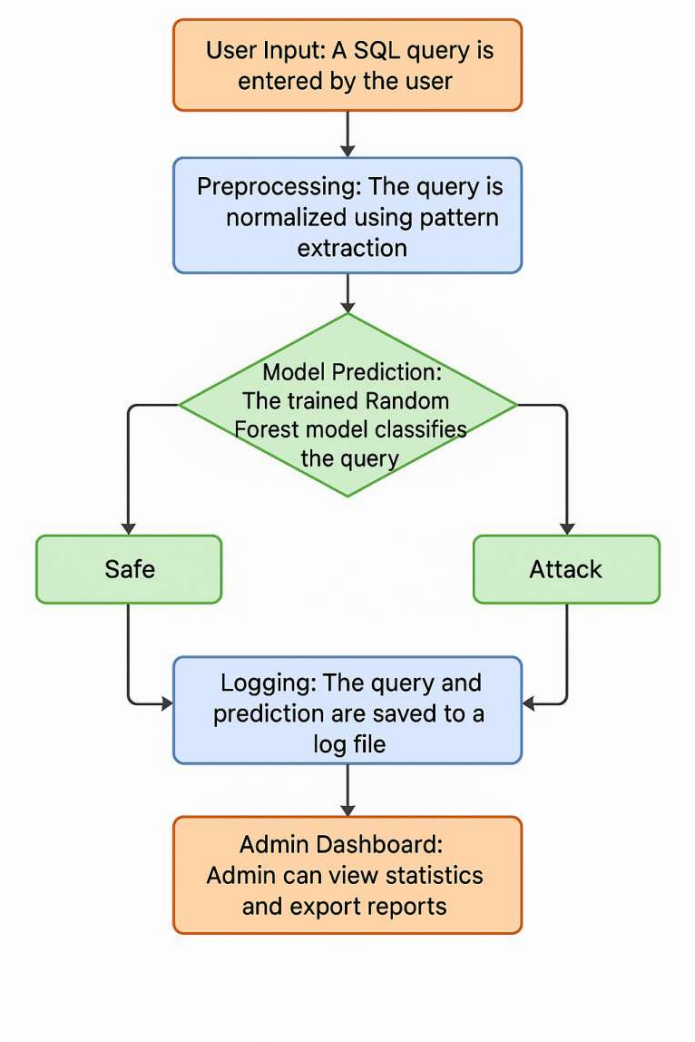


Figure : flow of the project

3.4 I/O Design

- **Input:** Raw SQL queries entered by the user.
- **Output:** A classification label ("Safe" or "Attack") is displayed on the screen.

Forms are designed using HTML and CSS for responsive design. Input fields are validated to avoid empty submissions.

3.5 Table Design

Though no SQL database is used, a logical representation of logs:
Log Table (in file-based form):

- ID (Auto Increment)
- Query (Text)
- Prediction (Safe/Attack)
- Timestamp
- IP Address

These logs can later be stored in a database for scalability.

Field Name	Data Type	Description
ID	Integer (Auto Increment)	Unique identifier for each log entry
Query	Text	The SQL query submitted by the user
Prediction	Varchar (Safe/Attack)	Result of model prediction (Safe or Attack)
Timestamp	Datetime	Date and time when the query was submitted
IP Address	Varchar	IP address of the user submitting the query

CHAPTER 4

IMPLIMENTATION & TESTING

4.1 Hardware and Software Requirements

Hardware:

- Minimum: Intel i3, 4GB RAM, 2GB disk space
- Recommended: Intel i5 or higher, 8GB RAM

Software:

- OS: Windows/Linux
- Python 3.8+
- **Required libraries**: Flask, sklearn, joblib, xhtml2pdf, pywebview

4.2 Installation Procedure

- Install Python 3.8 or above.
- Clone the project folder.
- Run pip install -r requirements.txt.
- Execute train_model.py to train the model.
- Start the desktop app using run_desktop.py.

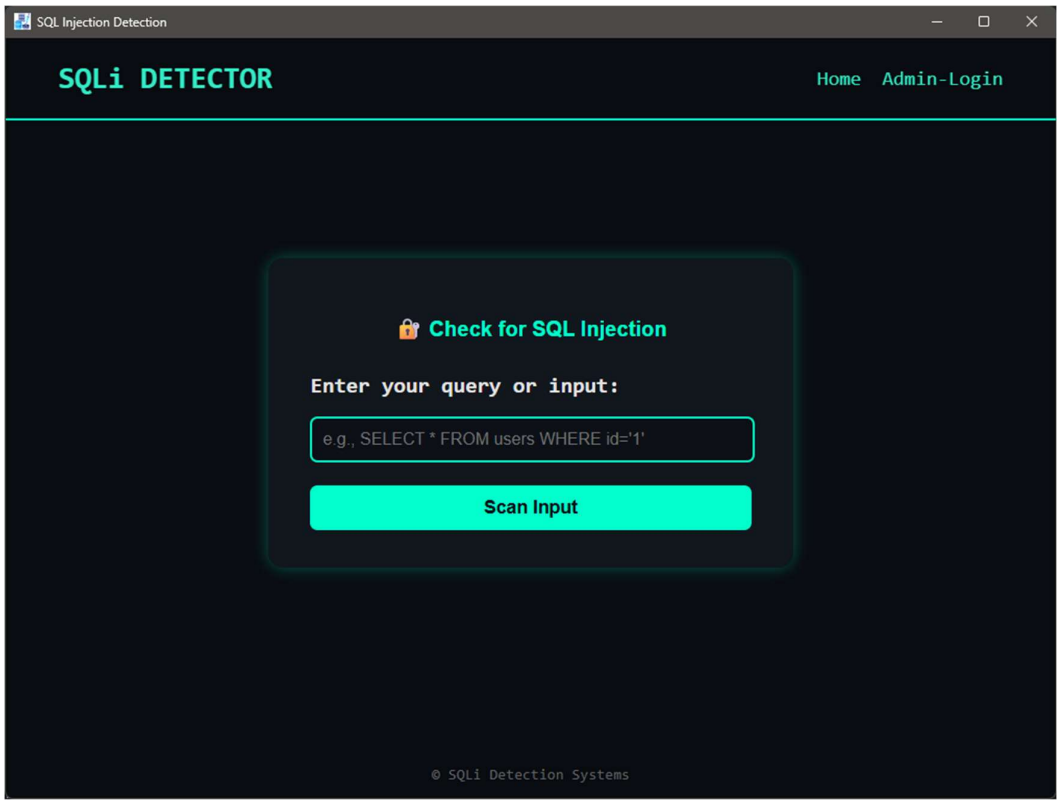
The GUI opens with the SQL Injection Detection interface.

4.3 Special Features of Python

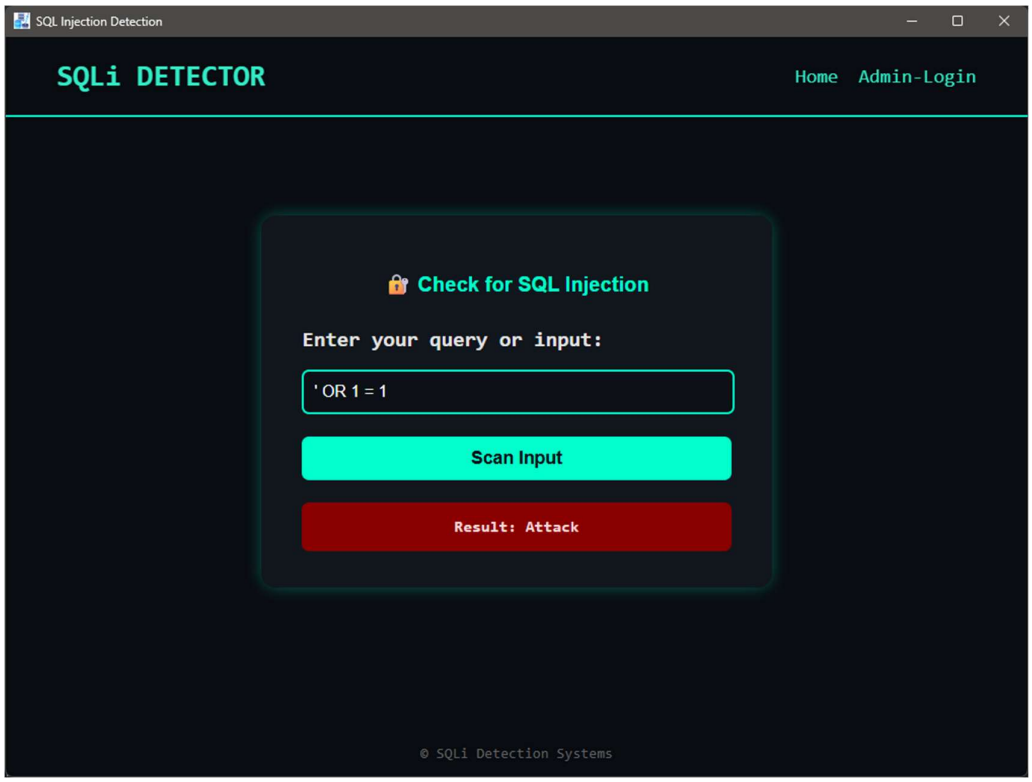
- Python's dynamic typing and easy syntax enable rapid development and prototyping, which is ideal for academic and real-time security applications.
- Extensive libraries like **Scikit-learn**, **Pandas**, **NumPy**, and **Joblib** support data preprocessing, model building, evaluation, and storage.
- The **Flask** framework provides a micro web server environment, allowing easy integration of the ML model into a real-time interface.
- Integration with **xhtml2pdf** enables effortless generation of downloadable PDF reports from HTML templates.
- Python supports **multithreading**, allowing the web server to run in the background while being wrapped into a GUI using **PyWebView**.
- Its wide community support and open-source nature make Python ideal for research and educational projects with minimal costs.

4.4 Screenshots

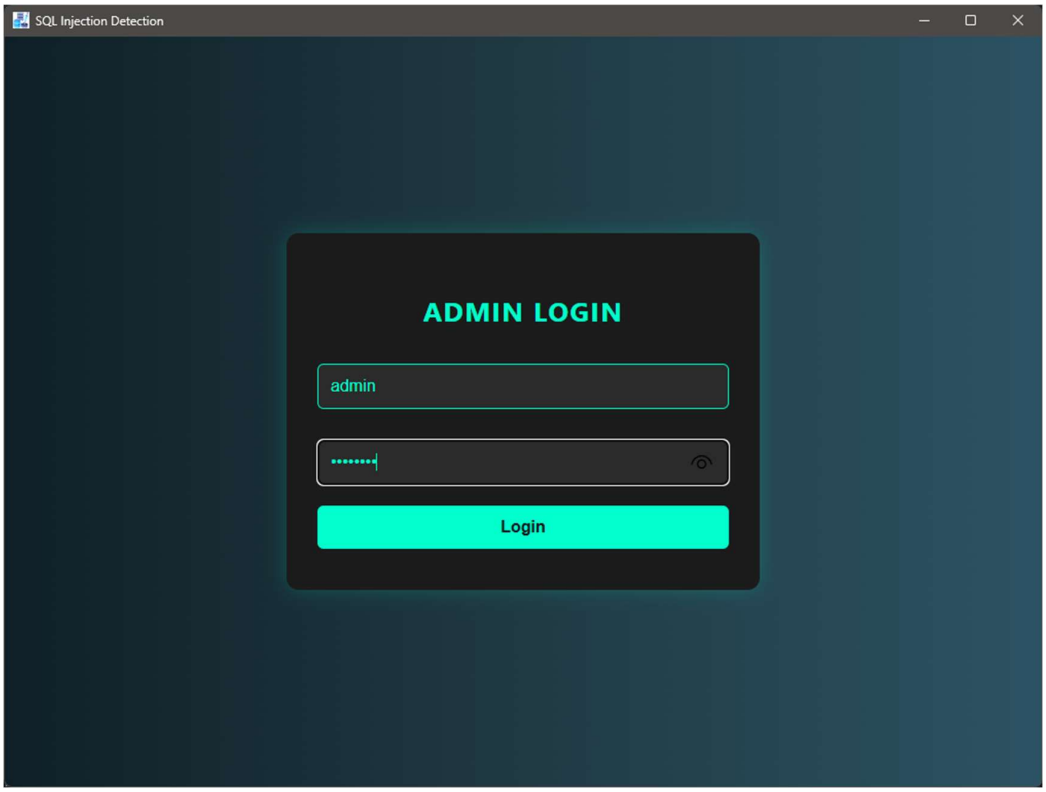
User Interface to Enter the SQL Query



Check for SQL Injection Detection



Login for Admin



Admin Dashboard

4.5 Unit testing

- Queries were individually passed through the frontend interface to verify end-to-end prediction flow.
- Known attack vectors such as ' OR '1'='1, '--, UNION SELECT, sleep(5) and information_schema were tested and flagged correctly.
- Valid SQL statements were used to ensure there are no false positives.
- Special cases including SQL encoding (%27 for ') and extra whitespaces were tested to verify preprocessing efficiency.
- The pattern_extractor.py module was tested separately to ensure normalization was consistent and accurate.
- Backend logic including logging and admin features was verified for correct functionality.
- The predict() function from the trained Random Forest model was tested with mock inputs to confirm accurate classification.

4.6 Validation Testing

- The dataset was split using an **80:20** ratio to ensure a representative test set.
- Accuracy achieved during testing was consistently between **93% and 95%**, indicating good generalization.
- Detailed classification metrics (precision, recall, F1-score) were generated using **Scikit-learn**'s classification_report function.
- The model showed high **recall** for the "Attack" class, proving its usefulness in identifying potentially dangerous queries.
- Testing was also performed on synthetic test cases with altered attack strings to test model adaptability.
- The system was tested on different machines to ensure consistency in prediction results across environments.
- Validation confirmed that the model does not overfit and maintains high performance across diverse SQL query inputs.

4.7 Experimentals Results

The model was trained on a cleaned dataset of 32,663 SQL queries (17,106 safe, 15,557 attack). Queries were vectorized using CountVectorizer and classified with a Random Forest model.

- Training samples: 26,130
- Testing samples: 6,533
- Training time: ~157 seconds
- Feature dimensions: 155,267

Test Results:

- Accuracy: 84.8%
- Safe Precision/Recall: 0.91 / 0.79
- Attack Precision/Recall: 0.80 / 0.91
- F1-Score (both classes): 0.85

Training Results:

- Accuracy: 99.95%
- All metrics (Precision, Recall, F1-score): 1.00 for both classes

CHAPTER 5

CONCLUSION AND FUTURE ENHANCMENT

5.1 Summery of Achievements

- Developed an end-to-end SQL Injection Detection System using machine learning techniques.
- Successfully trained and evaluated a Random Forest classifier with high accuracy and robustness.
- Built a responsive and easy-to-use interface with Flask for users to interact with the system.
- Wrapped the web application in a desktop GUI using PyWebView, improving portability and ease of deployment.
- Enabled query logging, time-stamping, and IP tracking for forensic and audit purposes.
- Provided an admin panel for real-time monitoring, viewing logs, visualizing attack statistics, and generating PDF reports.
- Achieved reliable validation results with minimal false positives and strong generalization on unseen queries.
- Structured the project to be modular and easily extendable for future upgrades and integrations.

5.2 Challenges faced during Project

- Cleaning and preprocessing a large and noisy SQL query dataset required custom handling of anomalies and inconsistencies.
- Dealing with inconsistent labeling and class imbalance posed challenges during model training.
- Extracting meaningful features from SQL queries demanded significant effort in pattern recognition and regular expression tuning.
- Ensuring consistent behavior of the PyWebView desktop GUI across operating systems like Windows and Linux was challenging.
- Managing concurrency between the Flask server and PyWebView runtime for seamless UI experience.
- Avoiding overfitting and reducing false positives without sacrificing detection accuracy.
- Ensuring proper export of data into well-formatted PDFs using the xhtml2pdf library.

5.3 Future Enhancement

- Integrate the system with real-time web application firewalls (WAFs) to block malicious queries live.
- Upgrade the detection model using deep learning techniques like LSTM or GRU to better capture sequential patterns.
- Introduce advanced logging capabilities such as geo-location tracking, login attempts, and user behavior analytics.
- Implement automatic periodic retraining of the model using newly collected data from logs.
- Convert the current system into a RESTful API to allow external applications and services to access the detection model remotely.

REFERENCES

- **Meharaj Begum A** and **Dr. Michael Arock**. (2021). ***Efficient Detection Of SQL Injection Attack (SQLIA) Using Pattern-based Neural Network Model***. Bio-informatics and Parallel Computing Lab, Department of Computer Applications, National Institute of Technology, Trichy. https://www.researchgate.net/publication/350936801
- **Scikit-learn Developers**. (2023). ***Scikit-learn: Machine Learning in Python***. https://scikit-learn.org/
- **Flask Documentation**. (2023). ***Flask Web Framework***. https://flask.palletsprojects.com/
- **OWASP Foundation**. (2023). ***OWASP Top 10 Security Risks***. https://owasp.org/www-project-top-ten/
- **Python Software Foundation**. (2023). ***The Python Programming Language***. https://www.python.org/
- **xhtml2pdf Documentation**. (2023). ***Convert HTML to PDF using Python***.https://github.com/xhtml2pdf/xhtml2pdf
- **SQL Injection Attack Prevention Techniques**. (2023).https://portswigger.net/web-security/sql-injection
- **PyWebView Documentation**. (2023). ***Build GUI for Python Web Apps***. https://pywebview.flowrl.com/

APPENDIX

Output of the log file

NOTE : since these outputs are from the training phase so the Prediction may be wrong.

SQL Injection Detection Logs

S.No	Input	Prediction	IP	Timestamp
1	' OR '1'='1	Attack	127.0.0.1	2025-06-26 13:21:50
2	' OR 1=1 --	Attack	127.0.0.1	2025-06-26 13:21:55
3	' OR 'a'='a	Attack	127.0.0.1	2025-06-26 13:22:02
4	* OR 1=1 #	Attack	127.0.0.1	2025-06-26 13:22:08
5	' UNION SELECT null, username, password FROM users --	Attack	127.0.0.1	2025-06-26 13:22:15
6	' UNION SELECT 1,2,3 #	Safe	127.0.0.1	2025-06-26 13:22:21
7	' UNION SELECT 1,2,3 #	Safe	127.0.0.1	2025-06-26 13:22:29
8	' UNION SELECT table_name, null, null FROM information_schema.tables --	Attack	127.0.0.1	2025-06-26 13:22:42
9	'; DROP TABLE users --	Attack	127.0.0.1	2025-06-26 13:22:50
10	'; EXEC xp_cmdshell('dir') --	Safe	127.0.0.1	2025-06-26 13:22:55
11	' OR 1=1 --	Attack	127.0.0.1	2025-06-26 13:23:07
12	' OR 'x'='x' /*	Attack	127.0.0.1	2025-06-26 13:23:13

278	INSERT INTO feedback (user_id, message) VALUES (5, 'Great service!');	Safe	127.0.0.1	2025-07-10 10:13:19
279	SELECT * FROM users WHERE username = 'admin' OR '1'='1';	Attack	127.0.0.1	2025-07-10 10:33:30
280	SELECT * FROM users WHERE username = 'admin';	Attack	127.0.0.1	2025-07-10 10:33:39
281	SELECT * FROM login WHERE user = 'admin' /* AND pass = '123'*/;	Attack	127.0.0.1	2025-07-10 10:33:43
282	' OR '1'='1	Attack	127.0.0.1	2025-07-10 10:33:46
283	hello world	Attack	127.0.0.1	2025-07-10 10:33:49
284	pritesh	Attack	127.0.0.1	2025-07-10 10:33:52
285	SELECT id FROM users WHERE username = 'admin';	Attack	127.0.0.1	2025-07-10 10:34:10
286	INSERT INTO feedback (user_id, comment) VALUES (1001, 'Nice UI!');	Safe	127.0.0.1	2025-07-10 13:31:38
287	DELETE FROM attempts WHERE try_count > 5;	Safe	127.0.0.1	2025-07-10 13:31:52
288	SELECT * FROM login WHERE email = 'admin' OR 'abc' = 'abc' --';	Attack	127.0.0.1	2025-07-10 13:32:07
289	SELECT * FROM login WHERE email = 'admin' OR 'abc' = 'abc' --';	Attack	127.0.0.1	2025-07-10 15:08:38
290	UPDATE users SET last_login = NOW() WHERE user_id = 42;	Safe	127.0.0.1	2025-07-10 15:08:54

BONAFIDE CERTIFICATE

This is to certify that the internship work titled **Train Ticket Confirmation Predictor** is a Bonafide record of the work done by

Rachit Namdev

(205124073)

in partial fulfillment of the requirements for the award of the degree of **Master of Computer Applications** of the **NATIONAL INSTITUTE OF TECHNOLOGY, TIRUCHIRAPPALLI,** during the year 2025-26.

Dr. S.R. Balasundaram
Project Guide

Dr. S. Domnic
Head of the Department

Project Viva-voice held on _____

Internal Examiner

External Examiner

Train Ticket Confirmation Predictor

An Internship report submitted in partial fulfillment of the requirements for the award of the degree of

MASTER OF COMPUTER APPLICATIONS

in

COMPUTER APPLICATIONS

By

Rachit Namdev

(205124073)

DEPARTMENT OF COMPUTER APPLICATIONS
NATIONAL INSTITUTE OF TECHNOLOGY
TIRUCHIRAPALLI-620015

July 2025

BONAFIDE CERTIFICATE

This is to certify that the project titled **Train Ticket Confirmation Predictor** is a Bonafide record of the work done by

Rachit Namdev

(205124073)

in partial fulfillment of the requirements for the award of the degree of **Master of Computer Applications** of the **NATIONAL INSTITUTE OF TECHNOLOGY, TIRUCHIRAPPALLI,** during the year 2025-26.

Dr. S.R. Balasundaram
Project Guide

Dr. S. Domnic
Head of the Department

Project Viva-voice held on _____

Internal Examiner

External Examiner

ACKNOWLEDGEMENT

I express my deep sense of gratitude to **Dr. G. Aghila**, Director, National Institute of Technology, Tiruchirappalli, for granting me the opportunity to undertake this project and for providing an environment that fosters innovation and academic growth.

I would like to extend my sincere thanks to **Dr. S. Domnic**, Head of the Department of Computer Applications, for his continuous encouragement, valuable insights, and for facilitating the resources and support necessary to carry out this work effectively.

I express my gratitude to my project guide, **Dr. S. R. Balasundaram**, Department of Computer Applications, whose expert guidance, constructive feedback, and unwavering support were instrumental in shaping this project. His mentorship has been a constant source of motivation and clarity, not only during the course of this project but throughout my academic journey at NIT Trichy.

I express my thanks to all the faculty members, and scholars of NIT Trichy for their critical advice and guidance to develop this project directly or indirectly.

ABSTRACT

Train travel in India often involves uncertainty, especially when passengers book tickets under waitlisted status. To address this issue, the ***Train Ticket Confirmation Predictor*** provides a machine learning-based solution that forecasts the likelihood of a waitlisted ticket getting confirmed. This web-based application uses historical train data and predictive modelling to help users make informed travel decisions.

The system takes into account various inputs such as train number, travel class, waitlist position, source and destination stations, and the gap between booking and journey dates. A trained Random Forest classifier, integrated with scikit-learn, processes these inputs to generate confirmation probabilities. The model has been trained on a structured dataset containing past booking patterns and ticket statuses.

Designed using Flask for the backend and standard web technologies for the frontend, the application offers a user-friendly interface featuring station and train autocomplete, dynamic train filtering, and multi-passenger support. It provides prediction results for the selected journey date and the following four days, enabling better travel planning.
The project emphasizes offline functionality, modular design, and future scalability—allowing easy integration of real-time data or advanced features. By combining machine learning with practical usability, the system enhances the decision-making process for Indian railway passengers.

TABLE OF CONTENTS

Title page no.

Bonafide certificate…… 2

Acknowledgement…… 3

Abstract…… 4

Table of Contents…… 5

Chapter 1 INTRODUCTION…… 7 - 8

- **1.1** Introduction …… 7
- **1.2** Problem Statement and Motivation…… 7
- **1.3** Scope of the Project…… 8

Chapter 2 LITERATURE SURVEY…… 9 - 10

- **2.1** Introduction to Train Ticket Confirmation Predictor…… 9
- **2.2** Prediction Techniques…… 9
- **2.3** Machine Learning for Prediction system …… 10
- **2.4** Summary of Existing System…… 10

Chapter 3 SYSTEM ANALYSIS…… 11 - 14

- **3.1** Existing System…… 11
- **3.2** Proposed System…… 11
- **3.3** System Requirements …… 12
- **3.4** Functional Requirements …… 13
- **3.4** Non-Functional Requirements…… 13
- **3.5** Feasibility Study…… 14

Chapter 4 System Design…… 15 - 18

- **4.1** Design Overview…… 15

- **4.2** Modular Break Down…….. 15
- **4.3** Hierarchical Flow Diagram…….. 17
- **4.4** Input and Output Design …… 18

Chapter 5 Implementation and Screen Shot…… 19 - 20

- **5.1** Development Environment and workflow…… 19
- **5.2** Technologies and Libraries Used…… 20
- **5.3** File Structure…… 20
- **5.4** Screen Shots…… 22

Chapter 6 Testing…… 23 -23

- 6.1 Unit Testing…… 23
- 6.2 Integration Testing…… 23

Chapter 7 Conclusion …… 24

Chapter 8 Limitations and Future Scope…… 25

References…… 26

CHAPTER 1

INTRODUCTION

1.1 Introduction of the Project

Train Ticket Confirmation Prediction is a machine learning–based system developed to help railway passengers estimate the likelihood of their waitlisted tickets getting confirmed. Instead of relying on guesswork or scattered online suggestions, this system offers a smart, data-driven approach by analysing multiple factors such as train number, travel class, booking and journey dates, waitlist position, and the time gap between booking and travel.

The system is especially useful for families, frequent travellers, and individuals making time-sensitive plans, as it helps them make informed decisions based on prediction percentages. Users can view confirmation chances not just for one passenger, but for up to four passengers across five consecutive journey dates, making it easier to plan alternatives if needed.

The model powering this system is trained using a real-world dataset and implemented using a Random Forest Classifier. The application is built with Python and Flask on the backend, and features a user-friendly frontend using HTML, CSS, and JavaScript, including features like train and station autocomplete. With a modular structure, the system is also ready for future enhancements, including integration with live railway data sources for even more accurate results.

1.2 Problem Statement and Motivation

In the Indian Railways reservation system, a significant number of train tickets are booked under the **waitlist (WL)** category due to high passenger demand and limited seat availability. Passengers with waitlisted tickets are often left uncertain about whether their ticket will be confirmed before the journey date. This uncertainty leads to inconvenience, anxiety, and poor travel planning—especially during peak seasons and emergencies.

There is a strong need for a **data-driven prediction tool** that can analyze real booking data and provide travelers with an estimated **confirmation probability**, enabling them to make better-informed decisions about their travel plans.

1.3 Scope of the Project

The scope of the Train Ticket Confirmation Prediction project is centred around predicting the likelihood of confirmation for waitlisted Indian Railways tickets. The system currently supports key functionalities such as multi-passenger prediction, five-day forecast from the selected journey date, and dynamic input validation including real-time station and train number suggestions. The prediction model is trained on historical booking data, taking into account parameters like train number, class, waitlist position, and the booking-to-journey date gap.

The project is designed to run as a lightweight web application that can be deployed locally, making it accessible without requiring advanced infrastructure. While the current implementation uses a static dataset and pre-trained model, the modular structure of the codebase allows for easy integration of live railway data APIs, additional classes, quotas, or even real-time seat availability in the future.

CHAPTER 2
RELATED WORK

2. Literature Survey

2.1 Introduction to Reservation Prediction Systems

Reservation and ticketing systems in railway transportation have traditionally been used for booking and managing travel records. However, these systems have historically lacked predictive capabilities. With the growing demand for railway travel and the limited availability of confirmed seats, passengers often face uncertainty regarding their waitlisted ticket status.

In recent years, the integration of artificial intelligence and machine learning has enabled a new wave of intelligent systems that can predict the probability of ticket confirmation based on historical trends and booking patterns. These models can analyse parameters like waitlist position, travel class, booking time, and seasonal demand to provide real-time insights into confirmation chances. Unlike conventional systems that only display static availability data, AI-driven prediction tools aim to empower passengers with informed decision-making and better planning.

2.2 Train Ticket Confirmation Prediction Techniques

Predicting the confirmation of train tickets, especially for waitlisted passengers, is a growing area of research due to the unpredictability and complexity of Indian railway bookings. Traditional systems provide only current availability and waitlist numbers, offering no foresight into whether a ticket will be confirmed before the journey date.

Recent approaches leverage machine learning and statistical models to make predictions based on historical data, including features such as waitlist position, booking date, journey date, travel class, and train route. These models are trained on large datasets containing past booking outcomes to classify tickets as likely to be confirmed, waitlisted, or cancelled.

The project utilizes a supervised machine learning model—Random Forest Classifier—for predicting ticket confirmation. Key features like train number, class, availability, waitlist, and booking-to-travel gap are encoded and fed into the model for accurate classification. The use of such models offers significant benefits for travelers, enabling informed decisions and improving user confidence in the booking process.

2.3 Machine Learning for Prediction Systems

Machine learning has revolutionized predictive systems across various domains, including healthcare, finance, and transportation. In the context of railway reservation systems, classification models like Random Forest, Decision Trees, and Logistic Regression are widely used for predicting outcomes based on historical and real-time data.

In this project, a Random Forest Classifier is used to predict the confirmation status of train tickets. The model considers various input features such as train number, class, availability, waitlist position, and the gap between booking and journey dates. These features are processed and fed into the model, which then predicts the likelihood of the ticket being confirmed.

The modular structure of the prediction pipeline allows easy updates with new data and model improvements. Although the model currently relies on offline historical data, the framework is designed to support integration with live APIs or real-time datasets in the future, making it a scalable and extensible solution for railway passengers.

2.4 Summary of Existing Systems

Existing train ticket booking systems, such as those offered by IRCTC and other travel platforms, typically provide basic information such as availability, waitlist position, and fare. However, they do not offer any predictive insight into the likelihood of ticket confirmation, leaving passengers uncertain—especially in cases involving long waitlists.
A few third-party tools attempt to estimate confirmation chances based on crowd-sourced data or limited historical trends, but they often lack transparency, accuracy, and adaptability. These systems also tend to be either oversimplified or overly dependent on real-time data access, which may not always be reliable or available.
The proposed Train Ticket Confirmation Prediction system addresses these gaps by using a trained machine learning model that works on historical data to offer accurate and interpretable predictions. It runs locally, ensures privacy, and provides a user-friendly, step-by-step interface—making it suitable for travellers who want quick insights without relying on external APIs or cloud-based platforms.

CHAPTER 3

System Analysis

3.1 Existing System

In the traditional railway reservation system, passengers rely on the Indian Railways' **PNR (Passenger Name Record) status** to track the confirmation of their waitlisted tickets. This system updates periodically based on cancellations, quota adjustments, and train charts, but it does not offer **predictive insights** into whether a ticket will be confirmed before chart preparation.

Most users have to guess or depend on forums, travel agents, or anecdotal experience to estimate their chances of confirmation. While a few online platforms attempt to provide probability-based predictions, these are often inaccurate due to lack of transparency in the underlying models, insufficient data sources, or over-reliance on third-party APIs. Additionally, such platforms may not allow users to test different scenarios (e.g., changing class, train, or travel date) to optimize their chances.

3.2 Proposed System

The **Train Ticket Confirmation Prediction System** is designed to overcome the limitations of traditional waitlist tracking by introducing an intelligent, AI-powered prediction model. Instead of relying solely on periodic PNR updates, this system uses machine learning techniques to forecast ticket confirmation probabilities based on historical data, booking trends, and various influencing factors. The system is divided into three key modules:

- **Train and Route Filtering Module**: This module validates the source and destination stations and filters trains that actually pass through both stations in the correct route order using real train route data. This ensures users only see relevant train options.
- **Prediction Engine**: This module uses a machine learning model trained on features such as train number, class, waitlist number, booking-to-journey date gap, and availability. The model is trained on a cleaned and pre-processed dataset using a Random Forest Classifier, which predicts the probability of confirmation for each passenger across multiple days.

- **Multi-Passenger and Multi-Day Forecasting Module**: This module allows users to input the number of passengers and initial waitlist number. It then generates predictions for each passenger over the next five days, accounting for slightly lower chances for subsequent passengers in the same booking.

The system runs locally using Flask and displays results on a web dashboard. It provides a fast, interactive, and user-friendly way to help travellers make informed decisions about their ticket bookings.

3.3 System Requirements

a. Hardware Requirements:

- Minimum:
  - Processor: Intel i3 8th gen or equivalent
  - RAM: 4 GB
  - Storage: 500 MB
  - Camera: Standard monitor or laptop screen with browser support
- Recommended:
  - Processor: Intel i7 / AMD Ryzen 5 or higher
  - RAM: 8 GB or more
  - Storage: 1 GB (for training larger datasets and storing logs)
  - GPU: NVIDIA with CUDA support (for model training)

b. Software Requirements:

- Operating System: Windows 10 / Linux / macOS
- Python 3.7 or more
- Libraries & Frameworks:
  - Flask (for web framework)
  - Scikit-learn (for ML model)
  - Pandas, numpy (for data processing)
  - Joblib (for model serialization)
  - HTML/CSS/JavaScripts (for frontend)
- Browser: Any modern browser (Chrome, Firefox, Edge) for using the web interface.

3.4 Functional Requirements

- **User input Interface:** The system must provide a multi-step form where users can enter journey details including source, destination, journey date, train selection, class, number of passengers, and waitlist number. **Pose Estimation:** It must accurately extract human pose key points using MediaPipe.
- **Train Route Filtering:** The application should accurately identify and display trains that run between the selected source and destination in the correct route order using internal dataset validation.
- **Prediction Engine:** The trained machine learning model should predict the ticket confirmation chances based on factors like train number, class, availability, waitlist number, and gap between booking and journey dates.
- **Data Validation:** Inputs such as journey date, waitlist number, and route logic (source before destination) must be validated before submission.
- **Autocomplete Support:** Source, destination, and train number inputs should support real-time autocomplete using predefined station and train datasets.
- **Frontend Display:** Results should be presented clearly on the result page with date-wise predictions and visual clarity for individual passenger confirmation rates.

3.5 Non-Functional Requirements

- **Real-time Performance:** The system should deliver prediction results quickly after form submission, with minimal processing delay for a smooth user experience.
- **Accuracy:** The machine learning model should maintain high accuracy in predicting ticket confirmation, especially for edge cases like high waitlists or short booking gaps.
- **Scalability:** The project should be designed in a modular manner, allowing future additions like quota-based predictions, dynamic data updates, or integration with live APIs
- **Usability:** The interface should be intuitive and easy to use for all users, including non-technical individuals, with clear form fields and visually understandable output.
- **Portability:** The application should run seamlessly on standard desktop environments and be portable across platforms with Python and browser support (e.g., Windows, Linux).

3.6 Feasibility Study

Technical Feasibility

All technologies used in the project are open-source and compatible with standard local systems. The machine learning model is built using Python libraries such as scikit-learn, pandas, and joblib, and is trained on structured .csv data. The web interface is developed using Flask, with JavaScript and HTML for the frontend, making it lightweight and easy to deploy on local machines without requiring high-end configurations or cloud services.

Economic Feasibility

The project has been developed without the need for any commercial licenses. All tools and frameworks used are freely available and community-supported. Model training and testing were carried out locally, and no paid services like cloud APIs or commercial datasets were used, ensuring zero development cost.

Operational Feasibility

Once set up, the system is easy to operate for any end-user through a simple web interface. It provides step-by-step input forms and displays prediction results clearly. The application runs efficiently on standard personal computers, making it highly suitable for educational use, demonstrations, and small-scale public utility services like ticket booking assistance.

CHAPTER 4

System Design

4.1 Design Overview

The **Train Ticket Confirmation Prediction** system is designed as a modular web application that guides users through a step-by-step process to predict the probability of train ticket confirmation. The architecture integrates user input collection, backend preprocessing, machine learning-based prediction, and result visualization in a seamless, user-friendly interface.

The system begins by taking inputs such as source and destination stations, journey date, selected train, class, and waitlist status. It validates the station sequence using historical route data, ensures date validity, and computes the gap between booking and travel date. These processed inputs are then passed to a trained **Random Forest Classifier**, which predicts the confirmation chances for each passenger over five consecutive journey dates.

The backend is built using **Flask**, while the frontend uses **HTML**, **CSS**, and **JavaScript**, including autocomplete support for station and train selection. The machine learning model is trained using a cleaned and pre-processed dataset in **pandas**, with label encoding for categorical features. Model training was conducted locally using scikit-learn and saved using **joblib** for quick deployment.

The design is lightweight, efficient, and structured to support future upgrades, such as integration with live APIs or mobile platforms.

4.2 Modular Breakdown

- **Input-Module**
  This module collects user inputs through a multi-step web interface. Users enter source and destination stations, journey date, train number, class, number of passengers, and the initial waitlist (WL) number. Autocomplete features aid in selecting valid station codes and train numbers

- **Route-Validation-Module**
  This component verifies that the selected train runs between the provided source and

destination stations in the correct sequence. It uses the train_data.csv dataset to check the route integrity based on intermediate station lists.

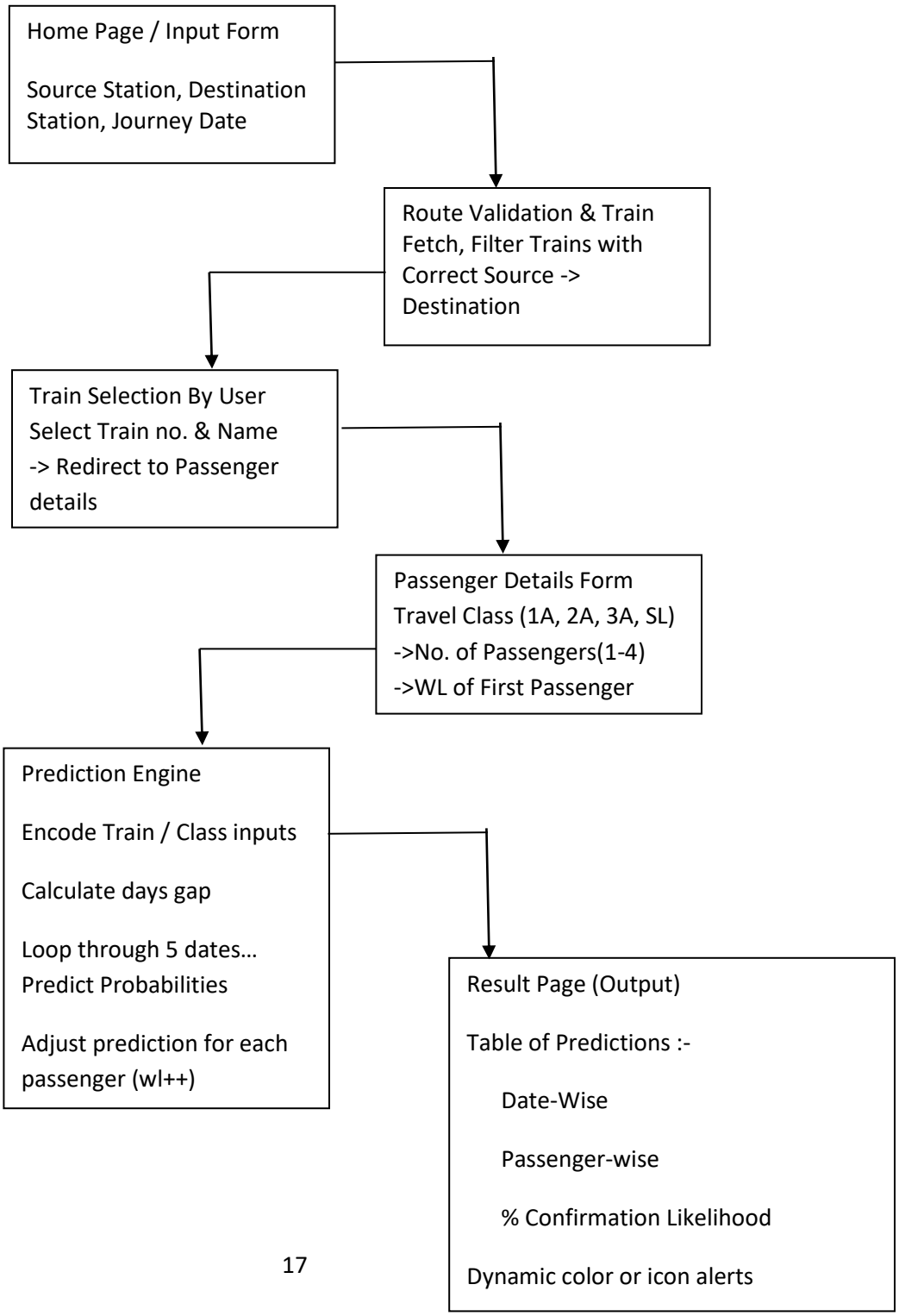
- **Preprocessing-Modul**
  This prepares the data for prediction by encoding categorical features (e.g., train number, class) using label encoders, and calculating the booking-to-journey gap. The data is structured into the same format used during model training.

- **Prediction-Module**
  A trained **Random Forest Classifier** predicts the ticket confirmation probability for each passenger. It returns predictions for five consecutive journey dates, adjusting subsequent passenger probabilities based on the first passenger's result and increasing WL numbers.

- **Frontend-Interface**
  Built using **Flask**, **HTML**, **CSS**, and **JavaScript**, this module displays forms for input and results. It includes client-side validation for WL limits and journey date gaps, and presents the predicted chances in a tabular, user-friendly format.

4.3 Hierarchical Flow Diagram (Description)

Here's a high-level description of the logical flow:



4.4 Input and Output Design

Inputs:

- User Input Form: Source Station, Destination Station, Journey Date
- Train Selection: Train Number
- Passenger Details: Travel Class, Number of Passengers, Waiting list of First Passenger
- Model Training Data: CSV FILE(Train no, Source, Destination, class, Availability, Waitlist, Booking Date, journey Date, Status)

Outputs:

Prediction Results:

- Date-wise confirmation probabilities (for the next 5 days)
- Passenger-wise prediction for each date
- Visualization in tabular format with color indicators (e.g., green for high chances, red for low)

Additional Outputs:

- WL increment per passenger shown dynamically
- Prediction accuracy (internally calculated during model training)
- Optionally downloadable result summary in CSV or JSON format

---

CHAPTER 5

Implementation

5.1 Development Environment and Workflow

The Train Ticket Confirmation Prediction project was developed using **Python 3.11**, leveraging open-source libraries and tools for both model training and web application deployment. The entire development was split into two main phases: **model training** and **web integration**.

Model Training Phase:

Model training was carried out on **Google Colab**, which provided a cloud-based environment with access to GPUs, enabling faster execution of data processing and machine learning tasks. The training dataset, stored in CSV format, was cleaned and preprocessed using **Pandas**. A **Random Forest Classifier** from **scikit-learn** was used as the primary prediction model. To ensure compatibility with machine learning algorithms, categorical fields such as train number, class, and status were encoded using **LabelEncoder**.

After training, the model and encoders were exported as .pkl files using **joblib** and downloaded for integration with the local system.

Web Application Phase:

The user interface and backend prediction system were built using **Flask**, a lightweight Python web framework. The project features a multi-step workflow where users:

1. Enter source, destination, and journey date.
2. Select a train from available options.
3. Enter ticket details such as class, number of passengers, and initial waiting list position.
4. View predictions for all passengers across five consecutive days.

All user inputs are validated, and the trained model generates real-time predictions based on current booking gap, availability, and waitlist position. The interface is styled using **HTML, CSS, and JavaScript**, and includes features like **autocomplete** for station codes and train numbers.

This clear separation between training and deployment allowed for efficient experimentation, modular upgrades, and scalable integration of future improvements—such as live data fetching or additional prediction models.

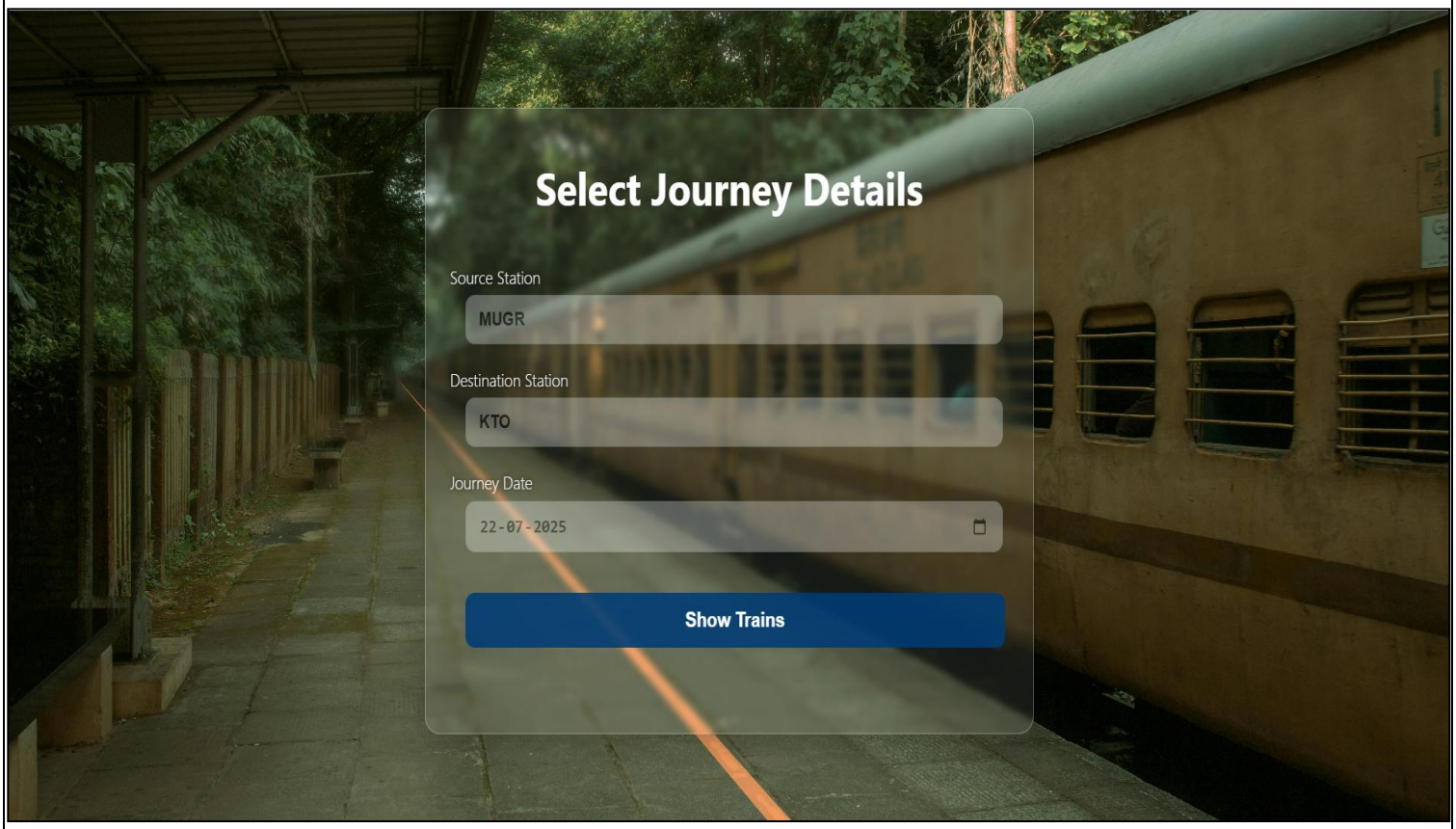
5.2 Technologies and Libraries Used

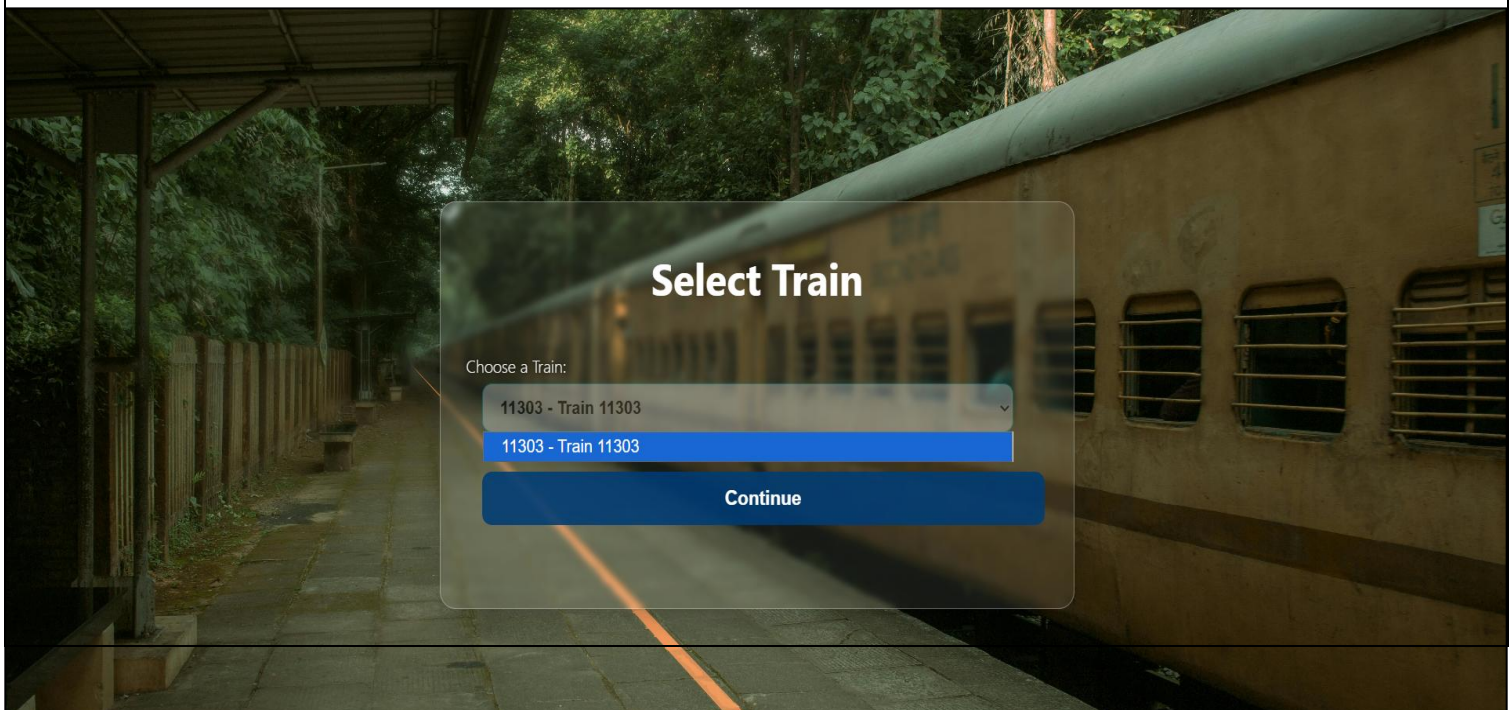
- **PYTHON 3.11 –** core programming language used for both for both backend logic and data processing.
- **Flask –** Lightweight web framework used to create a modular, multi-step web application.
- **Pandas –** For reading, preprocessing, and managing tabular data from csv files.
- **NumPy –** For numerical operations and efficient data manipulation during model preparation.
- **Scikit-learn –** Used to train the Random Forest classifier and perform data encoding.
- **Joblib –** For Saving and loading trained models and encoders in .pkl format.

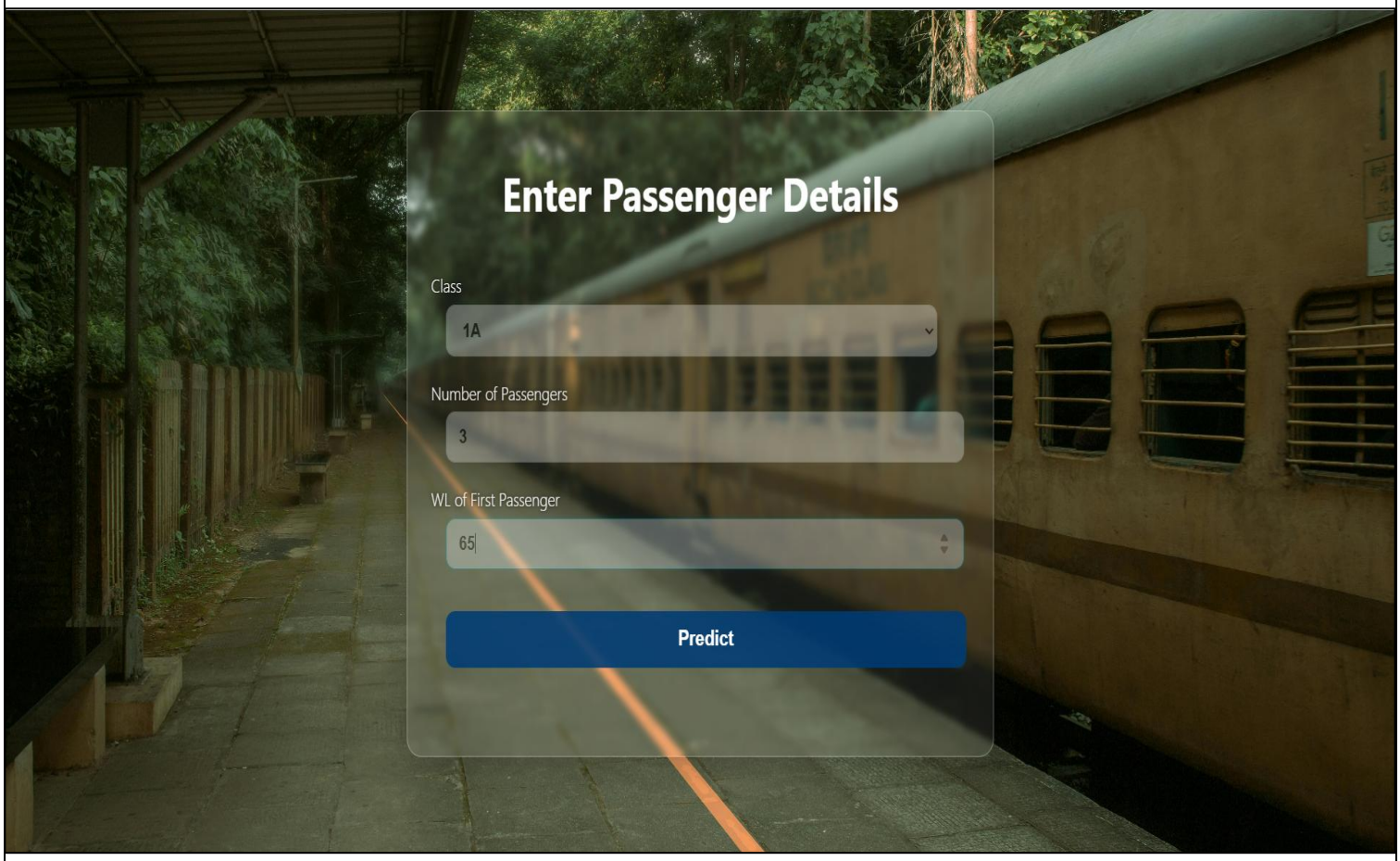
5.3 Directory and File Structure

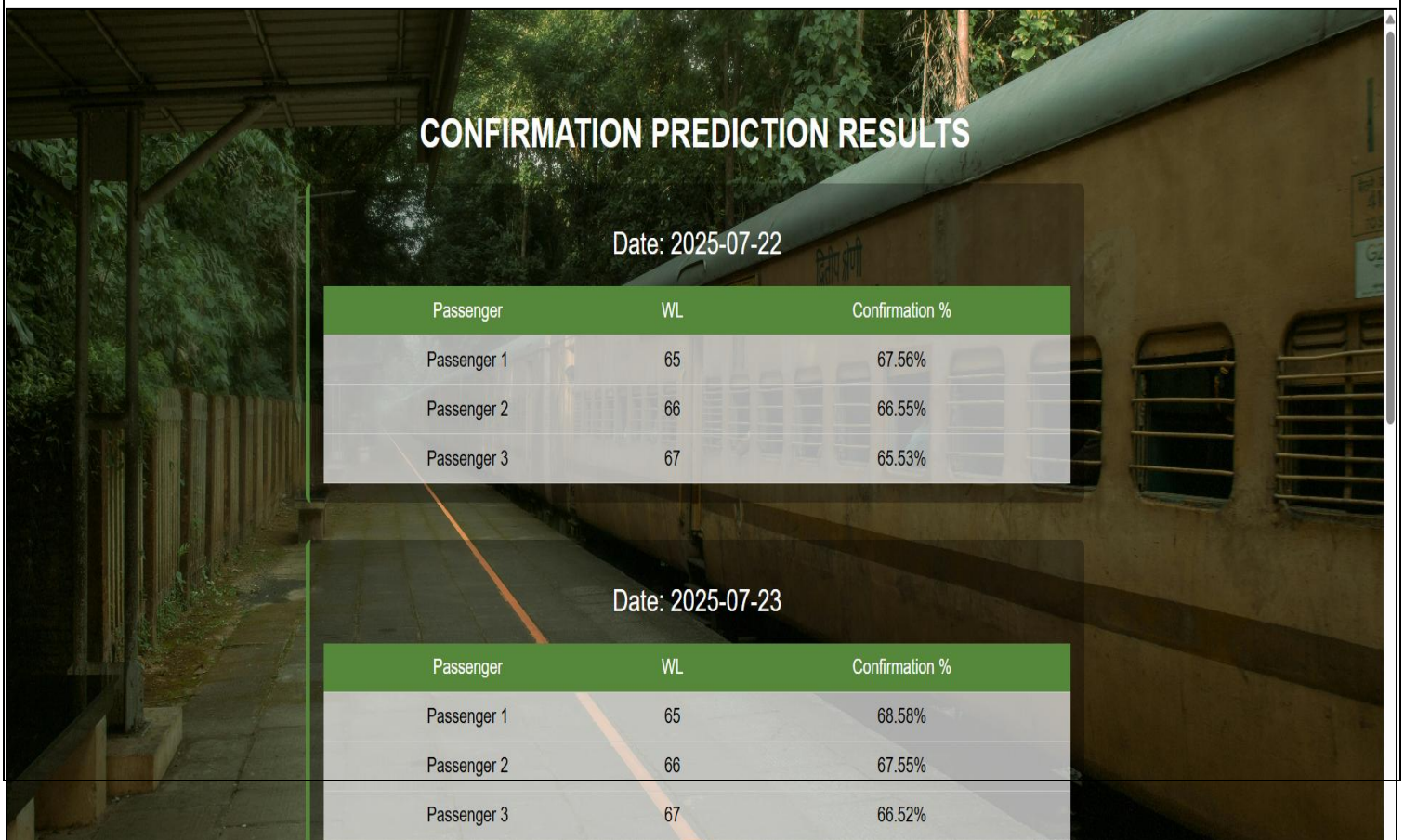
```
│
Train_Ticket_Confirmation_Pridictor/
├── app.py
├── train_model.py
├── train_data.csv
├── model.pkl
├── train_no_encoder.pkl
├── class_encoder.pkl
├── status_encoder.pkl
│
├── static/
│   ├── Background.jpg
│   ├── style.css
│   ├── stations.js
│   └── script.js
│
├── templates/
    ├── index.html
    ├── select_train.html
    ├── passenger_form.html
    └── result.html
```

5.4 Screen Shots









CONFIRMATION PREDICTION RESULTS

Date: 2025-07-22

Passenger	WL	Confirmation %
Passenger 1	65	67.56%
Passenger 2	66	66.55%
Passenger 3	67	65.53%

Date: 2025-07-23

Passenger	WL	Confirmation %
Passenger 1	65	68.58%
Passenger 2	66	67.55%
Passenger 3	67	66.52%

Chapter 6

Testing

6.1 Unit Testing

- Individual functions and modules such as data preprocessing, model prediction, and form validation were tested in isolation.
- Example: Verifying that the date gap calculation between booking and journey date returns accurate values.

6.2 Integration Testing

- Ensured that the model prediction logic, form inputs, and frontend-backend integration functioned smoothly together.
- Example: Testing the interaction between the Flask backend and HTML forms to ensure proper data flow.

Chapter - 7

Conclusion

The *Train Ticket Confirmation Prediction* project successfully showcases the application of machine learning in solving a real-world problem faced by railway passengers — estimating the likelihood of ticket confirmation based on available booking data. The system addresses the uncertainty commonly associated with waitlisted railway tickets and offers a data-driven solution to assist users in making informed travel decisions.

At the core of the project is a **Random Forest Classifier**, trained on a large dataset containing information such as train number, travel class, booking and journey dates, availability, and waitlist positions. The model was trained using Python and scikit-learn, and demonstrates reliable prediction accuracy in estimating confirmation probabilities.

The project features a multi-step user interface built with **Flask**, allowing users to:

- Select source and destination stations,
- Choose from a list of valid trains running between those stations,
- Enter travel details including class, quota, number of passengers, and initial waitlist,
- View prediction results for all passengers across five consecutive journey dates.

Special care was taken to ensure that train filtering respects route ordering and that prediction logic scales smoothly across multiple passengers, slightly adjusting probabilities based on waitlist progression. Additionally, the station and train input fields feature **autocomplete** to enhance usability.

Although the system currently relies on static historical data, its modular structure allows for easy future integration with live railway APIs or advanced machine learning models. The entire solution is lightweight, runs locally, and was developed using open-source tools, making it highly accessible and cost-effective.

In conclusion, this project successfully bridges machine learning with user-focused web development to deliver a smart, practical solution for travelers. It lays a strong foundation for further enhancement, such as integrating real-time data,

improving model accuracy, and expanding functionality to include personalized travel suggestions or alternative route predictions.

Chapter - 8

Limitations And Future Scope

Limitations:-

Despite its promising functionality, the current system has a few limitations. The prediction model is trained on a limited dataset that includes around 70–80 trains and approximately 250 stations. As a result, the application may not provide accurate predictions for all trains or station routes across the Indian Railways network. Additionally, the system relies on dummy data for ticket confirmation predictions, as official railway APIs do not provide access to real-time ticket confirmation data for free. This restriction limits the ability to integrate fully live and dynamically updating data into the prediction model. However, the architecture has been designed with extensibility in mind, allowing easy future integration with live data sources when accessible.

Future Scope:-

This project has strong potential for future enhancements and real-world deployment with broader functionality. One of the primary improvements would be integrating live railway data using official or reliable third-party APIs. This would allow the system to provide real-time predictions based on the latest availability and waitlist information, making the application far more accurate and practical.

Expanding the dataset is another key direction. Currently, the model is limited to a subset of trains and stations. With access to a larger and more diverse dataset covering more trains, classes, quotas, and routes, the model can be trained to provide significantly better generalization across the entire Indian Railways network.

In addition, user-based features such as tracking personal PNR status, saving frequent journeys, and receiving prediction-based alerts could greatly improve the system's usability. A mobile app version could also be developed for greater accessibility and convenience.

From a machine learning perspective, advanced models like ensemble methods or time-series forecasting techniques can be explored to improve prediction accuracy. Continuous learning using user feedback or actual ticket status (when available) could also help the model self-improve over time.

Overall, the system lays a solid foundation for a full-fledged smart ticket prediction assistant and can evolve into a powerful decision-making tool for railway passengers in the future.

REFERENCES

- **Indian Railway Data and Domain Understanding**
  - Indian Railways Official Website, https://indianrailways.gov.in
- **Machine Learning Libraries and Tools**
  - Scikit-learn: Machine Learning in Python. https://scikit-learn.org/
  - NumPy: Numerical Python. https://numpy.org/
  - Joblib: Serialization for Python. https://joblib.readthedocs.io/
  - Matplotlib & Seaborn: Visualizationtools. https://matplotlib.org/
- **Flask Web Frame Work**
  - Flask Documentation, https://flask.palletsprojects.com/